

# Programming Can Be Fun with Hy

Hy is an exciting new language that has surfaced just recently. Essentially, it is what Clojure is for Java—a Lisp implementation that has a very tight relationship with the host language, which is Python in this case. Hy allows developers to write their programs using its strong functional capabilities while leveraging the vast libraries of Python.



In this article, let's write a simple phone register program that stores people and their phone number in a SQLite database. The program will have a textual user interface. The source code is available at GitHub: <https://github.com/tuturto/register>.

## Pre-requisites

There are only a very few software components that we need. The first is Python. Version 2.7 or newer should work. The next important component is the Hy language itself. It can be installed with *Pip*:

```
sudo pip install Hy
```

The third component is the SQLite database. Installation instructions depend on the package manager you are using. With *Apt* for example, the command is as follows:

```
sudo apt-get install sqlite3
```

## A gentle introduction

Hy is a Lisp variant, which means that the programs might look quite different compared to those written with other languages—the biggest difference being that everything looks like a list and the operation is the first element in the list. For example, to calculate  $1 + 1$ , you need to type the following code:

```
(+ 1 1)
```

And the ever familiar 'hello world' would be written as:

```
(print "Hello World!")
```

To define a function that compares two numbers, you could use the following commands:

```
(defn comparison [x y]
  (cond ((= x y) (print x "and" y "are equal"))
        (< x y) (print x "is smaller than" y))
        (> x y) (print x "is bigger than" y))))
```

Don't worry if this looks strange or hard to understand, at first. After some practice, you'll be reading Lisp as easily as any other programming language. This article does not try to teach you everything about Hy. The goal is to get you familiar enough so that you can continue on your own.

## An example program

The example program will be a simple phone book application that can track people and their phone numbers. If you want to follow along, you can type the example code as you read into a file called *register.hy*. The order of code is often not that important, but the import form should be at the top and the main entry point should be at the bottom. There is also some Python code. You do not need to type that into the file; it is there to help understand how Hy works compared to Python.

Unlike Python, Hy is not particular about indentation. However, since there are a lot more parentheses around than normal, it is a good idea to group the code nicely and in a way that it is easily read. Tools like Emacs will be able to do this for you, automatically.

## Getting started with the database

An integral part of our phone book application is the database in which the data will be stored. Since we are using SQLite, creating a database, connecting it and creating the needed schema is easy. The first step is to import the *sqlite3* module so that we can call functions defined in it.

The next step is to define a few functions that we can later use to connect to the database and create the table where the